

Relational Algebra & Relational Calculus

1. Two Formal Languages for the Relational Model

	Relational Algebra	Relational Calculus
Nature	Procedural — specifies <i>how</i> to get the result, as a sequence of operations	Non-procedural (declarative) — specifies <i>what</i> result should look like, not how to compute it
Style	A sequence of operations (select, project, join, ...) forms an algebra expression	A predicate/condition describing the desired tuples

Why have both? Relational algebra is what a query optimizer actually reasons about and executes (a concrete sequence of steps it can transform/optimize). Relational calculus is closer to how SQL itself reads ("give me all tuples such that...") — SQL is largely calculus-flavored on the surface but gets translated into algebra-like execution plans underneath. Understanding both is exactly what lets you reason about "what SQL actually does under the hood."

PART 1 — Relational Algebra

2. Unary Operations (operate on a single relation)

Select (σ) — filters rows

$\sigma_{\langle \text{condition} \rangle}(R)$

- Chooses a subset of **tuples** from R that satisfy $\langle \text{condition} \rangle$.
- Properties:**
 - Degree** (number of attributes) of the result = degree of R (unchanged).
 - Cardinality** (number of tuples) can only **decrease or stay the same**: $|\sigma(R)| \leq |R|$.
 - Commutative**: $\sigma_1(\sigma_2(R)) = \sigma_2(\sigma_1(R))$ — order of successive selections doesn't matter.

Project (π) — filters columns

$\pi_{\langle \text{attribute list} \rangle}(R)$

- Chooses a subset of **attributes**, discarding the rest.
- **Properties:**
- Degree of result = number of attributes in `<attribute list>`.
- Cardinality $\leq$ original cardinality — because **duplicate elimination is automatically applied** (a relation is a *set*; projecting away distinguishing columns can make two previously-distinct tuples identical, and the duplicate is dropped).
- **NOT commutative** in general: $\pi_{\langle \text{list1} \rangle}(\pi_{\langle \text{list2} \rangle}(R)) \neq \pi_{\langle \text{list2} \rangle}(\pi_{\langle \text{list1} \rangle}(R))$ unless $\text{list2} \supseteq \text{list1}$.
- **Idempotent under a specific condition:** $\pi_{\langle \text{list1} \rangle}(\pi_{\langle \text{list2} \rangle}(R)) = \pi_{\langle \text{list1} \rangle}(R)$, if list2 contains all attributes in `list1`. (Projecting to a smaller list, then an even smaller one, is the same as projecting straight to the smallest one — as long as nothing needed later got dropped early.)

Special case: if `<attribute list>` contains a superkey of `R`, cardinality stays **exactly the same** (no duplicates possible, since a superkey already guarantees distinctness).

Worked example:

```
 $\pi$  Gender, Salary (Employee)
```

If two employees happen to share the same gender *and* salary, only **one** row appears in the result — but plain SQL allows duplicates unless you explicitly write `DISTINCT`; relational algebra's projection *always* removes duplicates by definition.

Rename (ρ)

```
 $\rho$  S( $B_1, \dots, B_n$ )(R)  - rename both the relation AND its attributes
 $\rho$  S(R)                 - rename only the relation
 $\rho$  ( $B_1, \dots, B_n$ )(R) - rename only the attributes
```

SQL equivalent: `... AS S`.

3. Set Operations (binary — require union compatibility)

Two relations `R` and `S` are **union-compatible** if: (1) they have the **same degree**, and (2) the ***i*-th attribute of R and the *i*-th attribute of S come from the same domain**, for every `i`.

Why? Each tuple in the result must "look the same" from both sources — you can't meaningfully union a "Name" column with a "Salary" column, even if they happen to have the same data type.

Operation	Symbol	Meaning	Notes
Union	$R \cup S$	Tuples in R, or S, or both	Duplicates removed; degree of result = same as R/S (they must match); commutative, associative
Intersection	$R \cap S$	Tuples in both R and S	Duplicates removed; commutative, associative
Set Difference	$R - S$	Tuples in R but not in S	Not commutative ($R - S \neq S - R$ in general)

SQL equivalents: `UNION`, `INTERSECT`, `EXCEPT` (or `MINUS` in some dialects).

4. Cartesian Product ($\times$)

$R \times S$

- Combines **every** tuple of R with **every** tuple of S — degree of result = degree(R) + degree(S); number of tuples = $|R| \times |S|$.
- On its own, almost always meaningless** — most tuple combinations are unrelated garbage.
- Common, useful pattern:** follow a Cartesian product with a σ (select) to keep only the combinations that are actually *related* — this pattern is so common it has its own shorthand operator: **Join**.

5. Join — the "select-after-Cartesian-product" shorthand

Mathematically: $\text{Join} = \text{Cartesian Product} + \text{Select}$.

Types of Join

Theta Join (θ -join) — join condition can use **any** comparison operator ($=, <, >, \geq, \leq, \neq$).

$R \bowtie (R.\text{salary} > S.\text{salary}) S$

Equi-Join — a special case of theta join where the condition uses **only** $=$.

$R \bowtie (R.\text{dept_id} = S.\text{dept_id}) S$

Natural Join ($\bowtie$, no explicit condition written) — a special case of equi-join where:

- The join condition is **implicit**: match on **all** attributes with the **same name** in both relations.

- The **duplicate attribute column is automatically removed** from the output (unlike a plain equi-join, which keeps both copies of the joined columns).

Worked example:

Students(SID, Name, DeptID)			Departments(DeptID, DeptName)	
1	Alice	D1	D1	CS
2	Bob	D2	D2	EE
3	Charlie	D3		

Equi-Join (`Students ⋈ Students.DeptID=Departments.DeptID Departments`):

SID	Name	DeptID	DeptID	DeptName
1	Alice	D1	D1	CS
2	Bob	D2	D2	EE

(Notice `DeptID` appears twice — once from each side; Charlie/D3 is missing because there's no matching `Departments.DeptID = D3`.)

Natural Join (`Students ⋈ Departments`):

SID	Name	DeptID	DeptName
1	Alice	D1	CS
2	Bob	D2	EE

(Only one `DeptID` column — the duplicate is automatically eliminated. Same matching rows, cleaner output.)

Outer Joins — keeping unmatched tuples

Regular ("inner") joins **drop** any tuple that has no match on the other side (Charlie was dropped above, since D3 has no matching department). Outer joins preserve them, padding the missing side with NULLs:

Join	Keeps unmatched tuples from...
Left Outer Join (<code>R ⋈⋈ S</code>)	R (left side)
Right Outer Join (<code>R ⋈⋈ S</code>)	S (right side)
Full Outer Join (<code>R ⋈⋈ S</code>)	Both sides

Worked example — Left Outer Join (`Students ⋈⋈ Departments`):

SID	Name	DeptID	DeptName
1	Alice	D1	CS
2	Bob	D2	EE
3	Charlie	D3	NULL

← Charlie is kept, DeptName padded with NULL

6. Division (÷)

$R \div S$ = tuples of R that are associated with ALL tuples of S

- Used for "**for all**"/"**every**" style queries.
- **Requires** $\text{attributes}(S) \subseteq \text{attributes}(R)$, and every attribute of S must also be in R , for the division to be meaningful.

Worked example: find students who have taken **every** course listed in `Course`.

Student_Course(SName, CName)		Course(CName)
S1	A	B
S1	B	
S2	B	
S3	A	
S3	B	

Student_Course $\div$ Course = { S1, S3 }

(Both S1 and S3 have taken every course in `Course` — namely, `A` and `B`. S2 only took `B`, so it's excluded.)

7. Generalized Projection & Aggregate Functions

- **Generalized projection** — lets projection compute values, not just select existing attributes: $\pi_{F_1, F_2, \dots, F_n}(R)$, where each F_i can be an arithmetic expression or a constant, not just a bare attribute name.
- **Aggregate function operator** — $\langle \text{grouping attributes} \rangle \bowtie \langle \text{function list} \rangle(R)$, computing summary values (COUNT, SUM, AVG, etc.) per group.

Worked example:

```
Dno ⋈ COUNT(Ssn), AVERAGE(Salary) (Employee)
```

→ produces, for each department, the count of employees and their average salary — directly analogous to SQL's `GROUP BY` + aggregate functions.

PART 2 — Relational Calculus

8. Tuple Relational Calculus (TRC)

Structure: $\{ t \mid \text{Condition}(t) \}$ — "the set of all tuples t such that $\text{Condition}(t)$ holds," where t is a **tuple variable** ranging over some relation.

Worked example: "All employees with salary > 50000":

```
{ t | Employee(t) AND t.salary > 50000 }
```

Projected to just the name:

```
{ t.fname | Employee(t) AND t.salary > 50000 }
```

Quantifiers, free & bound variables

- **$\exists$ (Existential)** — "there exists some..." — $\exists x (P(x))$ is true if *at least one* value of x makes $P(x)$ true.
- **$\forall$ (Universal)** — "for all..." — $\forall x (P(x))$ is true if *every* value of x makes $P(x)$ true.
- **Free variable** — not inside any quantifier; can appear in the result.
- **Bound variable** — appears inside a quantifier's scope; used only to help express the condition, doesn't itself appear in the result.

Worked example: "Find employees who work in the Research department":

```
{ e.fname, e.lname | Employee(e) AND  
  ( $\exists d$ )(Department(d) AND d.dname = "Research" AND e.dno = d.dnumber) }
```

Here e is free (its fields appear in the result); d is bound (only used inside the $\exists$ to express the condition).

Transformations between $\exists$ and $\forall$ (De Morgan-style equivalences)

```
( $\forall x$ )( $P(x)$ )  $\equiv$  NOT ( $\exists x$ )(NOT  $P(x)$ )  
( $\exists x$ )( $P(x)$ )  $\equiv$  NOT ( $\forall x$ )(NOT  $P(x)$ )
```

These let you rewrite a "for all" condition as a "there does not exist a counter-example" condition, and vice versa — genuinely useful, since some queries are far more natural to express with one quantifier than the other.

Safe Expressions

A **safe** expression is guaranteed to produce a **finite** set of tuples. An **unsafe** expression could, in principle, produce an infinite result.

Classic unsafe example:

```
{ t | NOT (Employee(t)) }
```

This literally means "every tuple that is *not* an employee" — an unbounded, infinite set (any tuple of the right shape not currently in the `Employee` table qualifies), so it's rejected as unsafe. Practical relational calculus implementations restrict expressions to guaranteed-safe forms.

9. Domain Relational Calculus (DRC)

Similar to TRC, but variables range over **single attribute (domain) values**, not entire tuples.

$$\{ x_1, x_2, \dots, x_n \mid \text{COND}(x_1, x_2, \dots, x_n, x_{n+1}, \dots, x_{n+m}) \}$$

- $x_1 \dots x_n$ — **free** variables, appearing in the result.
- $x_{n+1} \dots x_{n+m}$ — **bound** variables, appearing only within the condition.

Types of atoms in a DRC condition:

1. **Relational atom** — $R(x_1, \dots, x_j)$ — true if $\langle x_1, \dots, x_j \rangle$ is an actual tuple in R .
2. **Comparison atom** — $x_i \text{ op } x_j$ (comparing two variables).
3. **Comparison with a constant** — $x_i \text{ op } c$ or $c \text{ op } x_i$.

Worked example: "Find the birthdate and address of the employee named 'John B. Smith':"

$$\{ u, v \mid (\exists q)(\exists r)(\exists s)(\exists t)(\exists w)(\exists x)(\exists y)(\exists z) \\ (\text{Employee}(q, r, s, u, v, t, w, x, y, z) \text{ AND } q = \text{"John"} \text{ AND } r = \text{"B"} \text{ AND } s = \text{"Smith"}) \}$$

Shorthand form (a common convention — write out known constants directly, and only bind unused/irrelevant positions):

$$\{ u, v \mid \text{EMPLOYEE}(\text{"John"}, \text{"B"}, \text{"Smith"}, u, v, w, x, y, z) \}$$

Here u, v are free (they're what we want returned — birthdate and address); everything else is bound (used just to match the right tuple).

Interview Questions With Answers

Q1. Why is projection not commutative in general, but selection is?

Answer: Selection filters rows based on a condition, and applying two independent row-filtering conditions in either order produces the same final set of rows — the conditions don't interact with each other's evaluation. Projection removes columns, and once a column is dropped, any subsequent operation referencing that column becomes impossible or meaningless — so projecting to a smaller attribute list first and then trying to project to a list that included a now-missing attribute simply can't recover that information, making the order in which you apply successive projections matter (specifically, you can only go from a larger attribute list to a subset of it, not add attributes back).

Q2. Why does projection sometimes reduce the number of tuples in the result, while selection never increases it?

Answer: Selection only ever keeps a subset of the original tuples unchanged (based on a condition) or discards them — it never creates new distinct tuples, so cardinality can only decrease or stay the same. Projection can reduce cardinality because a relation is a mathematical set, and eliminating some attributes' values from a tuple can make two previously-distinct tuples (which only differed in the now-removed attributes) become identical — the automatic duplicate elimination that follows then merges them into one, reducing the tuple count.

Q3. Why is Cartesian product almost always used together with a select condition in practice, and what's the name for this common combination?

Answer: Cartesian product blindly pairs every tuple of one relation with every tuple of another, producing mostly meaningless combinations that have no real-world relationship to each other (e.g., pairing every employee with every department regardless of whether that employee actually works there). Following it with a select condition that keeps only the combinations satisfying some meaningful relationship (like matching foreign key to primary key) is what makes the result useful — and this specific "Cartesian product then select" pattern is common enough that it has its own dedicated operator: the **Join**.

Q4. What's the key difference between an equi-join and a natural join, given they're closely related?

Answer: Both restrict the join condition to equality comparisons, but a natural join goes further in two ways: it implicitly matches on *all* attributes sharing the same name between the two relations (no explicit condition needs to be written), and it automatically removes the duplicate copy of the matched attribute(s) from the output. An equi-join requires the join condition to be written explicitly and keeps both copies of the joined columns in the result (even though they're guaranteed equal for matching rows) — a natural join is essentially "the equi-join you'd write by default, with the redundant duplicate column cleaned up automatically."

Q5. Why does an inner join drop unmatched tuples, and what specific problem do outer joins solve?

Answer: An inner join is fundamentally select-after-Cartesian-product — a tuple with no matching partner on the other side simply never satisfies the join condition in any pairing, so it never appears in the Cartesian product subset that survives the selection, and is silently dropped. This becomes a problem whenever you specifically want to know about entities that *don't* have a match (e.g., "which students aren't assigned to any department yet") — outer joins solve this by explicitly preserving the unmatched tuples from one or both sides, padding the missing side's columns with NULL rather than dropping the row entirely.

Q6. Explain what the division operation computes, and why it requires the divisor's attributes to be a subset of the dividend's.

Answer: Division $R \div S$ returns the values from R's remaining attributes that are associated, in R, with **every** tuple currently in S — it's the relational algebra way of expressing "for all"/"every" style queries (e.g., "students who have taken every course in the Course table"). It requires $\text{attributes}(S) \subseteq \text{attributes}(R)$ because the operation works by checking, for each candidate value in R's non-shared attributes, whether its associated set of S-attribute values in R fully covers all of S — this comparison only makes sense if S's attributes are actually a subset of what R has to offer for that comparison in the first place; there'd be nothing to meaningfully match against otherwise.

Q7. Why is $\{ t \mid \text{NOT}(\text{Employee}(t)) \}$ considered an unsafe expression in relational calculus?

Answer: This expression describes the set of every possible tuple (of the appropriate shape/domain) that is *not* currently a row in the Employee relation — since the domain of possible values for most attributes (e.g., all possible strings, all possible integers) is unbounded, this describes a potentially infinite set of tuples, not a finite, computable result. Relational calculus requires expressions to be "safe" — guaranteed to always produce a finite result — specifically to ensure queries are actually computable in practice; an expression whose truth is defined purely by *absence* from a table, without any other bounding condition, structurally cannot guarantee finiteness.

Q8. What's the fundamental structural difference between Tuple Relational Calculus and Domain Relational Calculus?

Answer: In TRC, variables range over entire **tuples** of a relation (e.g., t ranges over rows of Employee, and you refer to $t.\text{salary}$, $t.\text{fname}$, etc., as fields of that tuple variable). In DRC, variables range over **individual domain values** directly (e.g., a separate variable for each attribute position — salary, name, etc.) rather than grouping them into one tuple-shaped variable — DRC conditions then use relational atoms like $\text{Employee}(q,r,s,\dots)$ to assert that a specific combination of individual domain values forms an actual tuple in the relation, rather than referring to fields of an already-bound tuple variable.

Q9. Scenario: You need to find all employees who work on every project that the "Research" department controls, but you only know relational algebra, not SQL. Which operation is the natural fit, and how would you structure the query at a high level?

Answer: The Division operation is the natural fit, since "every project" is exactly the "for all" pattern division is designed to express. At a high level: first, project a relation of $(\text{SSN}, \text{PNumber})$ pairs representing which employee works on which project (from a Works_On-style relation) — this becomes the dividend R. Then, build a relation of just the PNumber s controlled by the Research department (selecting Department by name, then finding its controlled projects) — this becomes the divisor S, containing only the PNumber attribute (matching the subset requirement for division). Computing $R \div S$ then yields exactly the SSNs of employees who are associated, in the Works_On

data, with *every* project number that appears in S — precisely the employees who work on every Research-controlled project.